



## 62-61.2 – Pratiques de développement

### Integration Testing

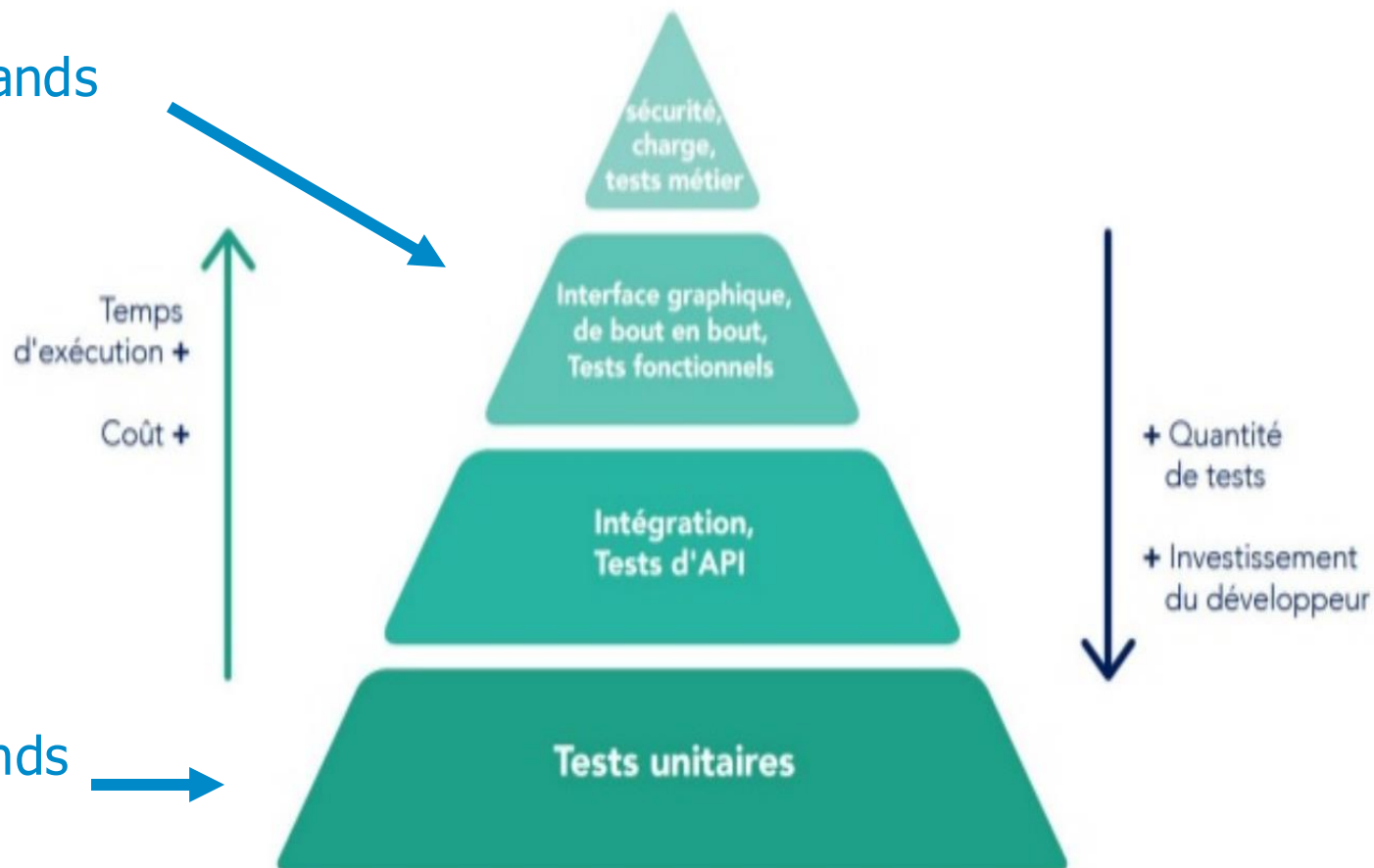
[antoine.induni@he-arc.ch](mailto:antoine.induni@he-arc.ch)

d'après Arnaud Geiser

## The Test Pyramid

ATDD stands  
here !

TDD stands  
there !



# Critères d'acceptation

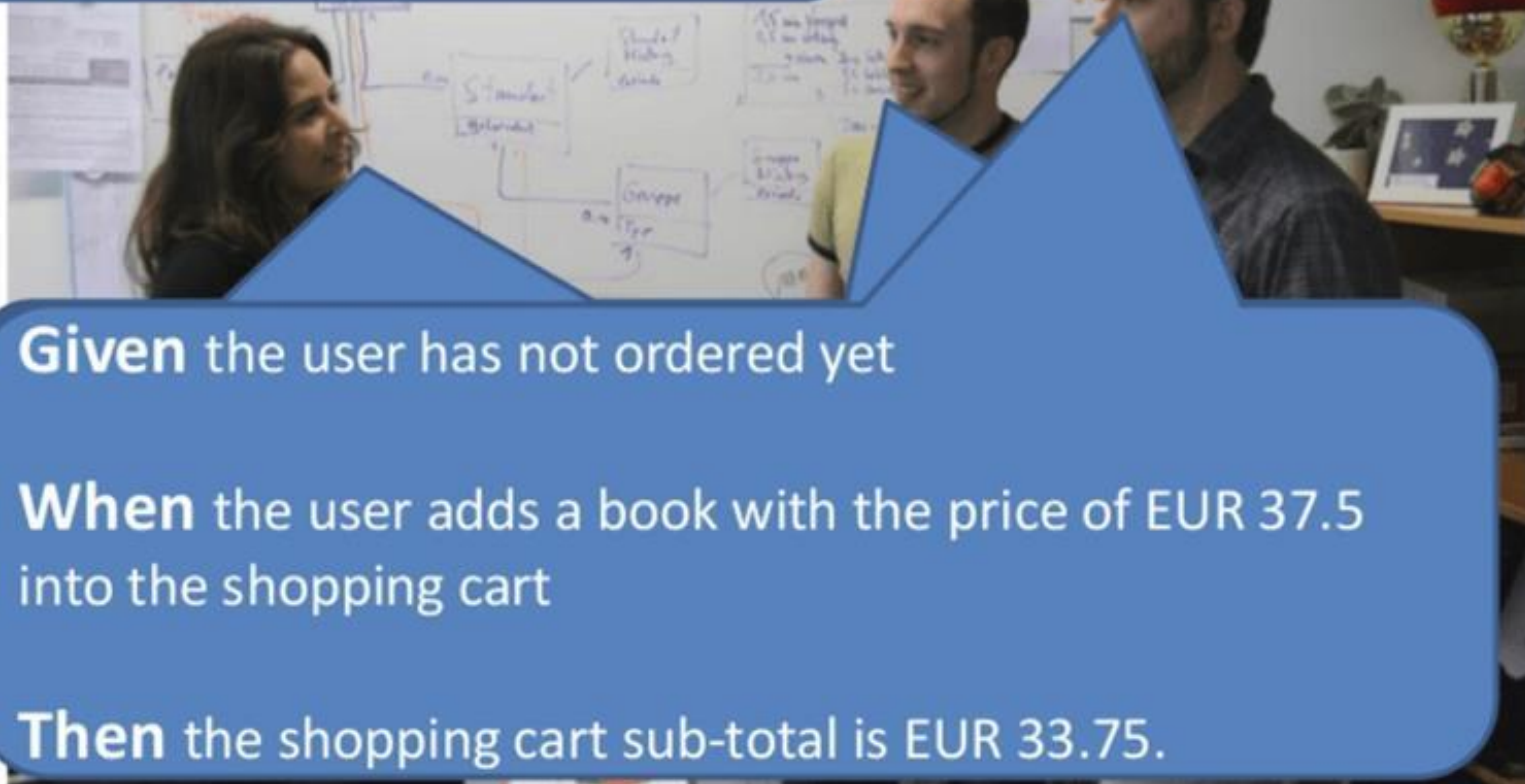
We would like to encourage new users to buy in our shop.  
Therefore we offer 10% discount for their first order.

```
public void CalculateDiscount(Order order)
{
    if (order.Customer.IsNew)
        order.FinalAmount =
            Math.Round(order.Total * 9/10);
}
```

Register as "bart\_bookworm"  
Go to "/catalog/search"  
Enter "ISBN-0955683610"  
Click "Search"  
Click "Add to Cart"  
Click "View Cart"  
Verify "Subtotal" is "\$33.75"

# Critères d'acceptation

We would like to encourage new users to buy in our shop.  
Therefore we offer 10% discount for their first order.



## Critères d'acceptation - Gherkin

**Given (Etant donné que)** : décrit le contexte initial

**When (Lorsque)** : décrit l'action faite par l'utilisateur à partir du contexte initial

**Then (Alors)** : décrit le comportement attendu suite à l'action faite par l'utilisateur

**Étant donné** que je suis sur la page de connexion

**Lorsque** je saisis mon nom d'utilisateur et mon mot de passe de manière incorrecte et que je clique sur "Connexion"

**Alors** le message d'erreur "Désolé, nom d'utilisateur ou mot de passe incorrect" s'affiche.



## Critères d'acceptation – Gherkin - And

Scenario: All done

**Given** I am out shopping

**And** I have eggs

**And** I have milk

**And** I have butter

**When** I check my list

**Then** I don't need anything

# Critères d'acceptation – Gherkin - Background

Feature: Multiple site support

Only blog owners can post to a blog, except administrators,  
who can post to all blogs.

## **Background:**

Given a global administrator named "Greg"

And a blog named "Greg's anti-tax rants"

And a customer named "Dr. Bill"

And a blog named "Expensive Therapy" owned by "Dr. Bill"

Scenario: Dr. Bill posts to his own blog

Given I am logged in as Dr. Bill

When I try to post to "Expensive Therapy"

Then I should see "Your article was published."

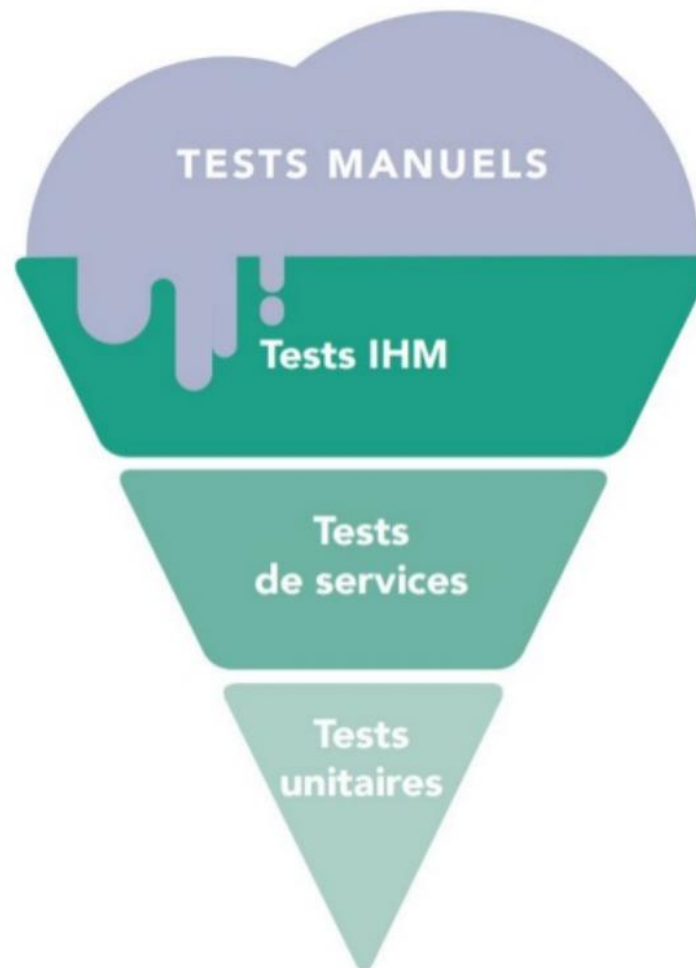
Scenario: Dr. Bill tries to post to somebody else's blog, and fails

Given I am logged in as Dr. Bill

When I try to post to "Greg's anti-tax rants"

Then I should see "Hey! That's not your blog!"

## The Reversed Test Pyramid





## Actions, calculations & data

### Data

Facts about events

*Examples: The email address a user gave us, the dollar amount read from a bank's API*

### Calculations

Computations from input to output

*Examples: Finding the maximum number, answering who is eligible for allocations?*

### Actions

Depend on how many times or when it's run

Examples: Sending an email, reading from/writing to the database

## Actions, calculations & data

### Data

Facts about events

*Not need to be tested*

### Calculations

Computations from input to output

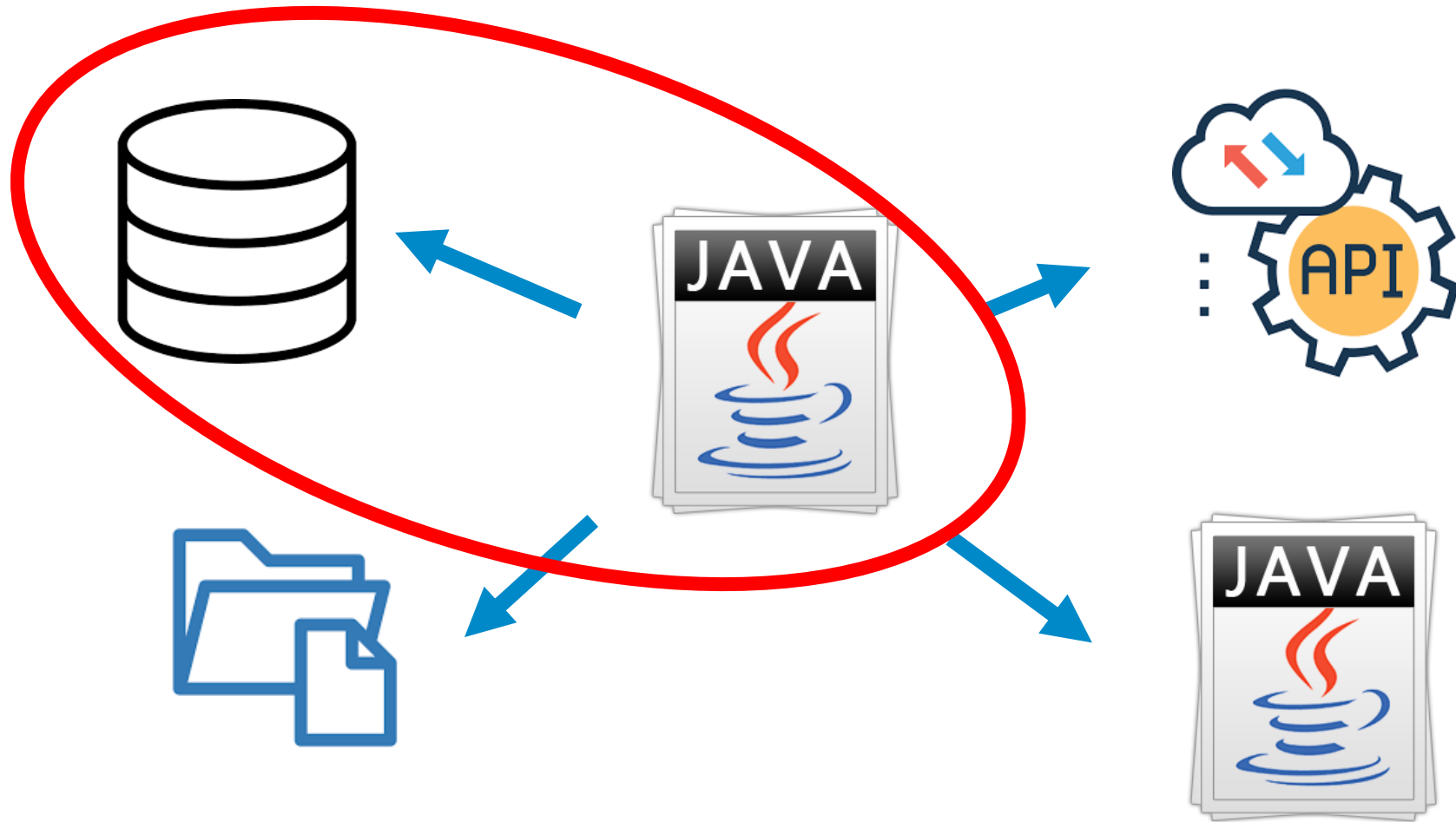
*Easy and simple to unit test!*

### Actions

Depend on how many times or when it's run

*Hard/Impossible to unit test, integration tests required*

## Integration Testing



## Integration Testing - Database

1. Set the database to a known state }
2. Run the system under test (SUT)
3. Check assertions

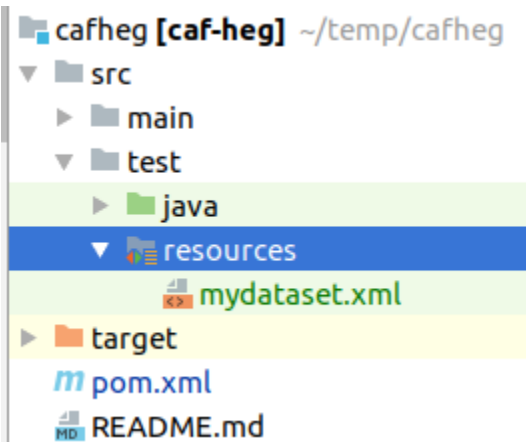


AssertJ

Fluent assertions for java

## DBUnit – XML File

```
<dataset>  
  <EMPLOYEE ID="1" NOM="Geiser" PRENOM="Arnaud"/>  
  <EMPLOYEE ID="2" NOM="Geiser" PRENOM="Chloé"/>  
  <EMPLOYEE ID="3" NOM="Geiser" PRENOM="Aurélié"/>  
  <EMPLOYEE ID="4" NOM="Geiser" PRENOM="Quentin"/>  
  <EMPLOYEE ID="5" NOM="Geiser" PRENOM="Matthieu"/>  
</dataset>
```



in **src/test/resources**

## DBUnit – API

### 1. Create a database connection

```
IDatabaseConnection connection = new DatabaseConnection(jdbcConnection);
```

### 2. Load a dataset (XML file) into memory

```
IDataset dataset = new FlatXmlDataSetBuilder()  
    .build(getClass().getClassLoader().getResourceAsStream("mydataset.xml"));
```

### 3. Clean and insert the dataset into the database

```
DatabaseOperation.CLEAN_INSERT.execute(connection, dataset);
```



## DBUnit – API

### 4. Retrieve expected data

```
ITable expectedTable = dataset.getTable("EMPLOYEE");
```

### 5. Retrieve actual data

```
ITable actualData = connection.createQueryTable(  
    "result_name",  
    "SELECT * FROM EMPLOYEE");
```

### 6. Assert

```
assertEquals(expectedTable, actualData);  
//or  
assertEqualsIgnoreCols(expectedTable, actualData, new String[] { "id" });
```

```
<dataset>
  <PERSON NAME="Charlie" LAST_NAME="Brown" AGE="42"/>
  <PERSON NAME="Bob" LAST_NAME="Doe" AGE="18"/>
  <PERSON NAME="Alice" LAST_NAME="Foo" AGE="23"/>
</dataset>
```

```
create table if not exists PERSON (
  ID int identity primary key,
  NAME varchar,
  LAST_NAME varchar,
  AGE smallint,
)
```

```

public class XmlDatabaseTest {

    private static final String JDBC_DRIVER = org.h2.Driver.class.getName();
    private static final String JDBC_URL = "jdbc:h2:mem:test;DB_CLOSE_DELAY=-1";
    private static final String USER = "sa";
    private static final String PASSWORD = "";

    @BeforeClass
    public static void createSchema() throws Exception {
        RunScript.execute(JDBC_URL, USER, PASSWORD, "schema.sql", UTF8, false);
    }

    @Before
    public void importDataSet() throws Exception {
        IDataset dataSet = readDataSet();
        cleanlyInsert(dataSet);
    }

    private IDataset readDataSet() throws Exception {
        return new FlatXmlDataSetBuilder().build(new File("dataset.xml"));
    }

    private void cleanlyInsert(IDataset dataSet) throws Exception {
        IDatabaseTester databaseTester = new JdbcDatabaseTester(JDBC_DRIVER, JDBC_URL, USER, PASSWORD);
        databaseTester.setSetUpOperation(DatabaseOperation.CLEAN_INSERT);
        databaseTester.setDataSet(dataSet);
        databaseTester.onSetup();
    }

    @Test
    public void findsAndReadsExistingPersonByFirstName() throws Exception {
        PersonRepository repository = new PersonRepository(dataSource());
        Person charlie = repository.findPersonByFirstName("Charlie");

        assertThat(charlie.getFirstName(), is("Charlie"));
        assertThat(charlie.getLastName(), is("Brown"));
        assertThat(charlie.getAge(), is(42));
    }

    @Test
    public void returnsNullWhenPersonCannotBeFoundByFirstName() throws Exception {
        PersonRepository repository = new PersonRepository(dataSource());
        Person person = repository.findPersonByFirstName("iDoNotExist");

        assertThat(person, is(nullValue()));
    }

    private DataSource dataSource() {
        JdbcDataSource dataSource = new JdbcDataSource();
        dataSource.setURL(JDBC_URL);
        dataSource.setUser(USER);
        dataSource.setPassword(PASSWORD);
        return dataSource;
    }
}

```